



SAP Application Development (S/4 HANA)

Basic Concepts

Motivation

This chapter is about the basic concepts in ABAP programs. You will focus on data declaration as well as data manipulation. Furthermore, you will create a program which uses user inputs. In the last section you will use a BAPI to use external functionality which is not included into your program.

Prerequisites

Before starting the exercises you should successfully proceed through the *Introduction to SAP Programming* chapter.

Content

This chapter explains the basic concepts of ABAP programs. It focuses on the structure of data, the data manipulation and the functions. Furthermore you will learn how to use external program and functions in your programs.

Lecture notes

The fundamental understanding of the ABAP development in the SAP system is a prerequisite for the students. Students do not use the developments from previous chapters. However, all users need a developer key to be able to do the exercises. Please refer to the *Introduction to SAP Programming* chapter for further assistance on this topic.

- **Product:** All
- **Level:** Beginner
- **Focus:** Programming
- **Version:** 2.0
- **Author:** UCC Technical University of Munich

Task 1: Start Eclipse and open your project

Short description: Start Eclipse and open an existing project or create a new one

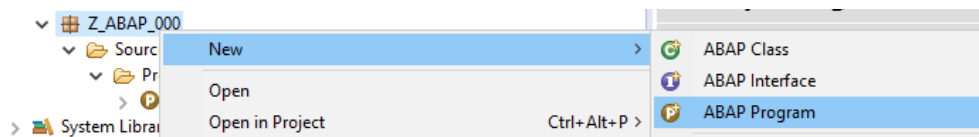
Start Eclipse and open the ABAP perspective. If an ABAP project exists please open this project. Else create a new ABAP project according to the *Introduction to SAP Programming* chapter. Please refer to your instructor for your username and your password. Ensure having your own package added to the favorites in Eclipse. If you don't have an own package, create a new one according to the mentioned chapter.

**Start
Eclipse**

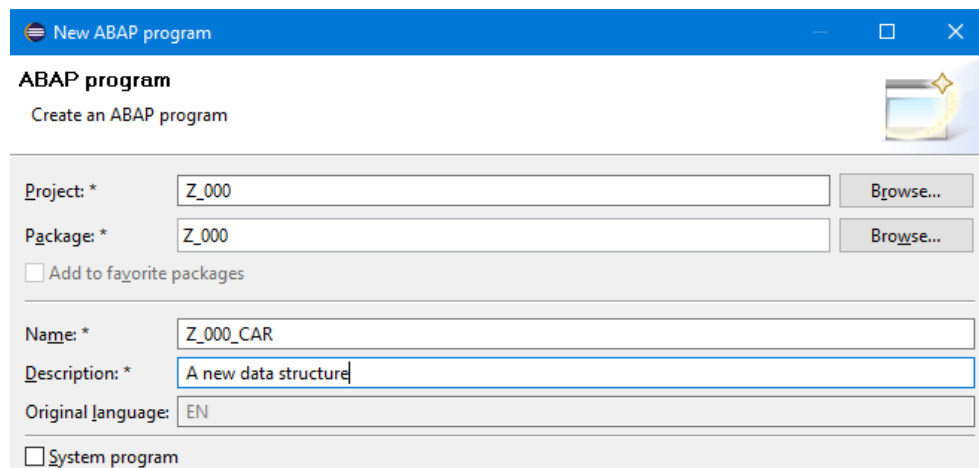
Task 2: Data declaration and manipulation

Short description: Use data declaration to declare a new data structure for your variables and use the debugger to get information about the variables during runtime

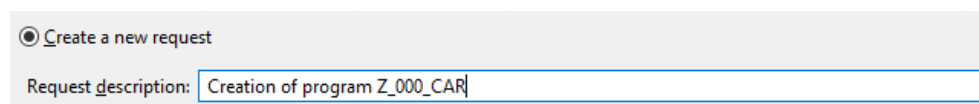
Create a new program called **Z_\$\$_###_CAR**. Please use the information provided by your instructor to replace \$\$ and ###. Perform a right-click on your package and select **New • ABAP Program**.



Enter the program name and a short description.



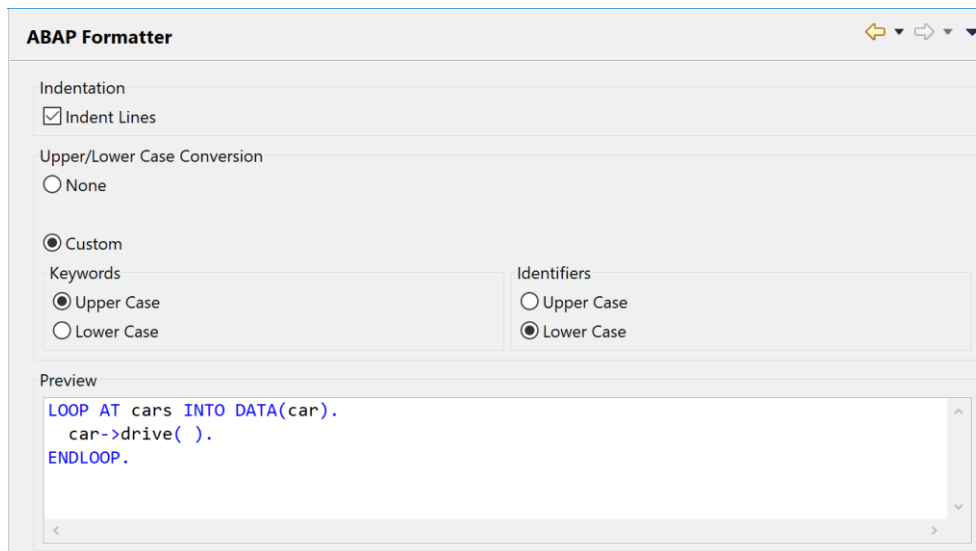
Create a new transport request for your program.



In order to properly format your source code you can use the ABAP formatter. In order to set up this feature right-click on your project in Eclipse and select **properties**. Use following navigation path:

ABAP Development • Editors • Source Code Editors • ABAP Formatter

and use following settings:



Now you create a new data structure called 'str_car'. This is done by using the 'TYPE' instruction. The new structure includes data about cars such as brand, model, mileage, fuel-tank capacity and fuel efficiency. There are different data types used in this structure. Brand and model are character types (**c**) whereas the mileage and fuel-tank capacity are of type (**i**). Fuel efficiency on the other hand is of type (**c**). You restrict the length of the brand and model to a maximum of 40 characters.

```

1  REPORT Z_000_CAR.
2
3  TYPES: BEGIN OF str_car,
4           brand(40)      TYPE c,
5           model(40)      TYPE c,
6           mileage        TYPE i,
7           fueltank_cap   TYPE i,
8           fuel_efficiency(6) TYPE c,
9       END OF str_car.
10
11 DATA coolcar TYPE str_car.
```

Hint:

Use the ABAP formatter to format your source code by pressing Shift+F1 or use the menu path:

Hint

Source Code • Format

In the next step you want to use the new structure to store some data inside. Therefore, you have to assign values to the structure. This can be done by using the name of the structure and the field names. It is mandatory to use spaces between the structure's name and the value.

Assign values

```

13 coolcar-brand = 'Mercedes'.
14 coolcar-model = 'SLR'.
15 coolcar-mileage = '10254'.
16 coolcar-fueltank_cap = '70'.
```

Finally, you want to prove if the variable declaration was successful. Therefore, we use the write instruction to write out the value of each structure field. This is done by using the 'WRITE /' instruction. The slash at the end of the write instruction indicates

WRITE /

that a new line should be opened after every output. So your complete source code should look similar to this:

```

1 REPORT Z_000_CAR.
2
3 TYPES: BEGIN OF str_car,
4         brand(40)      TYPE c,
5         model(40)      TYPE c,
6         mileage        TYPE i,
7         fueltank_cap   TYPE i,
8         fuel_efficiency(6) TYPE c,
9     END OF str_car.
10
11 DATA coolcar TYPE str_car.
12
13 coolcar-brand = 'Mercedes'.
14 coolcar-model = 'SLR'.
15 coolcar-mileage = '10254'.
16 coolcar-fueltank_cap = '70'.
17
18 WRITE: / 'Brand: ', coolcar-brand, / 'Model: ', coolcar-model, / 'Mileage: ',
19        coolcar-mileage, / 'Fuel-Tank Capacity: ', coolcar-fueltank_cap.

```

The result can be viewed after you saved, activated and run your program. It should look similar to this:

Save and activate

A new data structure

A new data structure

Brand: Mercedes
 Model: SLR
 Mileage: 10.254
 Fuel-Tank Capacity: 70

To get a first feeling for debugging your code you will now start the debugging mode by setting a so called breakpoint. Debugging will be explained in more detail later in this chapter.

Debugging introduction

To set the breakpoint perform a double click in the grey sidebar beneath the according line in your source code. Set the breakpoint at line 14. A blue dot should appear:

```

14 coolcar-model = 'SLR'.

```

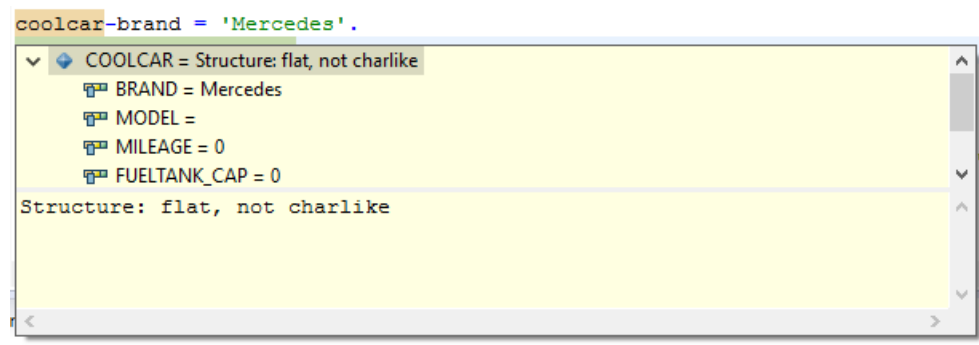
When running your program now you confirm the switch to the debugging perspective by clicking yes.

When you put your mouse-cursor now over your structure you can see the current state of your structure:

```

coolcar-brand = 'Mercedes'.

```

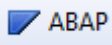


COOLCAR = Structure: flat, not charlike

- BRAND = Mercedes
- MODEL =
- MILEAGE = 0
- FUELTANK_CAP = 0

Structure: flat, not charlike

You will notice that the brand has been set but model, mileage, fuel-tank capacity and fuel efficiency haven't yet been set. Perform now a step over by pressing the F6 key or the  button. You will notice the marker indicating the current execution has moved one line down and the model has been set.

You can now press the resume button  or the F8 key. Change back into the ABAP perspective by pressing  and remove the breakpoint in your code by double-clicking on it.

Task 3: User input and control structures

Short description: Use selection screens and control structure to create a simple calculation program

Now we will add some calculation and user inputs to our program. You will calculate the fuel efficiency of your car by entering the trip mileage and the fuel consumed on your last trip. Since the calculator will be international you can choose if it should return l/100km or miles per gallon. You have to add according **parameters** and a variable for your result to your code – so add these two lines (11 and 13):

```
10
11 PARAMETERS: trip_f TYPE i, trip_m TYPE i, country TYPE c.
12
13 DATA result TYPE f.
14 DATA coolcar TYPE str_car.
15
16 coolcar-brand = 'Mercedes'.
```

The **PARAMETERS** instruction is used to handle the user input. Both operands are integer (i) variables whereas the country-selection is char (c) type with the length of 1.

PARAMETERS

Save, activate and run your program now. You will notice your program will ask you for the parameter-values. The SAP system automatically generates the 'Execute' button (F8). Unfortunately, you have not defined any actions or screen names yet.

To set the selection texts please right click on one of your parameters and select:

Open Others • Text Elements

The integrated SAP GUI will open now. Afterwards open the "Selection Texts" tab by clicking on it.

Changing screen names

Switch into the editing mode by clicking the  button. Change the screen names and click the save button  and the activate button  of the integrated SAP GUI. Finally, you can close the integrated SAP GUI.

Program			Z_000_CAR
Text Symbols			Selection Texts
List Headings			
Name	Text	Dictionary ...	
COUNTRY	Type in G (Germany) or U (USA)	☐	
TRIP_F	Fuel Consumed (Trip)	☐	
TRIP_M	Mileage (Trip)	☐	

Save, activate and run your program now and enjoy the results.

In the last step we want to implement the calculation. Your program uses two different calculation methods: one for the German expression (l/100km) and one for the US-American expression (mpg). It is assumed that the values for trip_f and trip_m are entered in units used in the respective countries (kilometers / liters or miles / gallons). To differentiate you will use a '**CASE**' instruction. You will also add an instruction to make the variable of type **f** more convenient to read.

CASE instruction

Your source code should look similar to this:

```

1 REPORT z_000_car.
2
3 TYPES: BEGIN OF str_car,
4     brand(40) TYPE c,
5     model(40) TYPE c,
6     mileage TYPE i,
7     fueltank_cap TYPE i,
8     fuel_efficiency(6) TYPE c,
9 END OF str_car.
10
11 PARAMETERS: trip_f TYPE i, trip_m TYPE i, country TYPE c.
12
13 DATA result TYPE f.
14 DATA coolcar TYPE str_car.
15
16 coolcar-brand = 'Mercedes'.
17 coolcar-model = 'SLR'.
18 coolcar-mileage = '10254'.
19 coolcar-fueltank_cap = '70'.
20
21 @CASE country.
22   WHEN 'G'.
23     result = trip_f / ( trip_m / 100 ).
24     WRITE result TO coolcar-fuel_efficiency EXPONENT 0.
25     WRITE: / 'Brand: ', coolcar-brand, / 'Model: ', coolcar-model, / 'Mileage: ',
26     coolcar-mileage, ' km', / 'Fuel-Tank Capacity: ', coolcar-fueltank_cap, ' liters', / 'Result: ',
27     coolcar-fuel_efficiency, 'l/100km'.
28   WHEN 'U'.
29     result = trip_m / trip_f.
30     WRITE result TO coolcar-fuel_efficiency EXPONENT 0.
31     WRITE: / 'Brand: ', coolcar-brand, / 'Model: ', coolcar-model, / 'Mileage: ',
32     coolcar-mileage, ' miles', / 'Fuel-Tank Capacity: ', coolcar-fueltank_cap, ' gallons', / 'Result: ',
33     coolcar-fuel_efficiency, 'miles per gallon'.
34 ENDCASE.

```

Save, check and activate your new program.

**Activate
everything!**

Task 4: Debugging

Short description: Use the debugger to get more information about a software bug and finally resolve this error

Begin with creating a new program called **Z_\$\$_###_SAFE**. Please use the information provided by your instructor to replace \$\$ and ###. Enter a short description and use your package and transport request.

ABAP program
Create an ABAP program

Project: * Z_000 Browse...

Package: * Z_000 Browse...

☐ Add to favorite packages

Name: * Z_000_SAFE

Description: * First debugging experience

Original language: EN

☐ System program

Now enter following source code to your program:

```

1  REPORT Z_000_SAFE.
2
3  * Define variables
4  DATA: usr_locked TYPE bapislockd.
5
6  * Perform BAPI call
7  CALL FUNCTION 'BAPI_USER_GET_DETAIL'
8    DESTINATION sy-sysid
9    EXPORTING
10     username = sy-uname
11    IMPORTING
12     islocked = usr_locked.
13
14 IF usr_locked = 'unlocked'.
15   WRITE 'Safe unlocked'.
16 ELSE.
17   WRITE 'Safe locked'.
18 ENDIF.

```

Save, check and activate your program. When running your program, you will notice following output:

First debugging experience

First debugging experience

Safe locked

When having a closer look at your program, you will notice the program performs a BAPI call which checks if the current user is unlocked or locked – e.g. because of too many wrong login attempts. So you are expecting the program to return 'Safe unlocked' instead of 'Safe locked', since your current user should be unlocked. In the current state the program seems to have a software fault. Now you want to search for the reason of this bug.

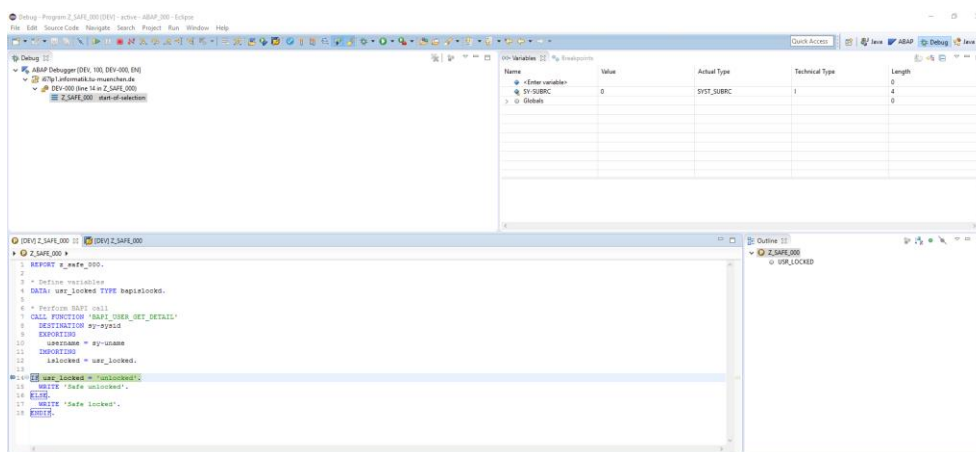
The fault seems to be located at the if-statement. So set a breakpoint there by performing a double click in the grey space right in front of the according line of code.

```

13
14 IF usr_locked = 'unlocked'.
15   WRITE 'Safe unlocked'.
16 ELSE.
17   WRITE 'Safe locked'.
18 ENDIF.

```

Now execute your program again. Eclipse wants to change now into the debugging perspective. Click yes to change the perspective.



Now have a closer look at this perspective. In the center of your screen you will find your source code with a small marker, telling you which line of code currently gets executed. Note that the currently marked line has not been executed yet. It will be executed when pressing the step-over button . Push this button now and have a look at the small marker. You will notice the marker has jumped to the code-line containing the else-branch.

```

13
14 IF usr_locked = 'unlocked'.
15   WRITE 'Safe unlocked'.
16 ELSE.
17   WRITE 'Safe locked'.
18 ENDIF.

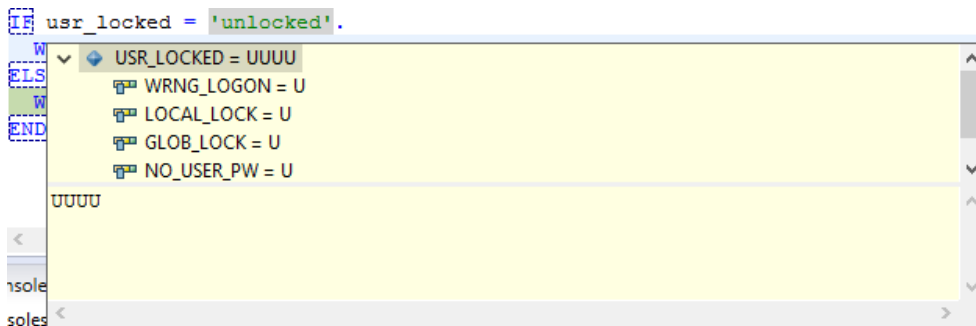
```

Now have a look why the else-branch has been chosen. This can be done by examining the current value of the variable 'usr_locked'. Now go to the 'Variables' window located in the right top edge of Eclipse and enter 'usr_locked' in the field currently stating '<Enter variable>' and press Enter. Afterwards click on the arrow in front of the variable to flip open the detailed view. The result should look similar to this:

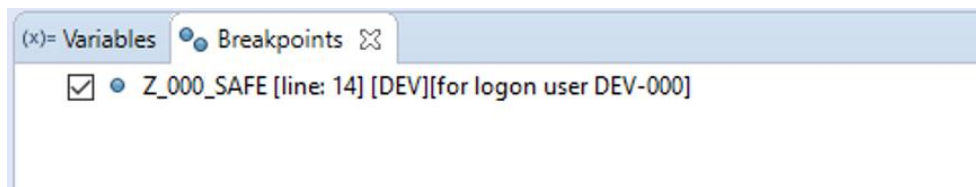
Name	Value	Actual Type	Technical Type	Length
<Enter variable>				0
USR_LOCKED	UUUU	BAPISLOCKD	Structure: flat, charlike	8
WRNG_LOGON	U	BAPIUSLOCK	C	1
LOCAL_LOCK	U	BAPIUSLOCK	C	1
GLOB_LOCK	U	BAPIUSLOCK	C	1
NO_USER_PW	U	BAPIUSLOCK	C	1
SY-SUBRC	0	SYST_SUBRC	I	4
Globals				0

UUUU

Alternatively, you can perform a mouse-over over the variable in your source code being displayed in the center of the screen.



You will notice the variable also contains details about the locking-state of the current user. It gives details if the user eventually has been locked due to too many wrong logins, if a local or global lock exists or if no password has been set for this user. The current value of the variable 'usr_locked' is also shown. It is not very hard to see that the value differs from the string 'unlocked', which is the reason for the software failure here. Since you now know the reason for the failure you can remove the breakpoint. Open the 'Breakpoints' window in the top right corner by clicking on the according tab next to the 'Variables' tab.



Perform a right click on the breakpoint and click 'Remove'. Now resume the execution of your program by pressing 'F8' or click the resume button . You can switch back now to the ABAP view by clicking the according button . Exit your program and close the internal SAP GUI. Now change the if-statement to the correct value you just received from debugging your application. Save, check and activate your program and enjoy the feeling of successfully having fixed a software fault.

